Igreja Assombrada — guia de estudo, parte 2

Os tópicos que faltam para explicar o projeto inteiro, com o código do próprio projeto

Este guia cobre o que ainda não foi estudado: **estrutura do programa, transformações e modelagem hierárquica, câmera e projeção, animação, colisão, mira, fases do jogo, HUD 2D e profundidade/transparência**. Malhas, texturas e iluminação já foram vistas (a iluminação tem um guia próprio), mas entram no resumo final.

Cada seção segue o mesmo formato: a ideia central em uma frase, a explicação para iniciante, o trecho real do código e o que ele faz. A **seção 10** traz respostas prontas, curtas e precisas, para o caso de o professor perguntar sobre uma parte específica.

Mapa do projeto: onde está cada coisa

Arquivo	O que tem dentro
`Main.cpp`	O programa em si: `main`, `setup`, `timer`, `display`, teclado, pausa/menu, mira.
`player/CameraMembroA.h`	Câmera em primeira pessoa: posição, yaw/pitch, WASD, mouse.
`render/ModelosMembroA.h`	Chão, paredes, teto, corredor, bancos, porta dupla, lustres.
`render/ModelosMembroB.h`	Altar, castiçais, confessionário.
`render/ModelosMembroC.h`	Cubo texturizado, vitrais, relógio de pêndulo, lamparina.
`render/Varinha.h`	A varinha apaga-velas.
`render/ShaderMembroB.h`	Shader de iluminação (Blinn-Phong), luzes e materiais.
`scene/LogicMembroC.h`	Colisão (AABB), portas, velas, mira (`notaMira`).
`scene/LogicMembroB.h`	Portas do confessionário (lógica e colisão).
`scene/GameplayMembroB.h`	Fases do jogo, o bicho, a caixa de texto (HUD).
`Textura.h`	Carregamento de imagens como textura.

1. Estrutura geral do programa

Ideia central: um programa GLUT não tem um laço escrito por você. Você **registra funções** (callbacks) e o GLUT chama cada uma na hora certa, para sempre.

Main.cpp — main()

```
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);   // 2 buffers de cor + buffer de profundidade
glutInitWindowSize(800, 600);
glutCreateWindow("Igreja Assombrada ...");

setup();                                  // roda UMA vez: shaders, texturas, cena

glutDisplayFunc(display);                 // "quando precisar desenhar, chame display"
glutReshapeFunc(reshape);                 // "quando a janela mudar de tamanho, chame reshape"
glutKeyboardFunc(keyboard);               // tecla apertada
glutKeyboardUpFunc(cameraTeclaUp);        // tecla solta
glutPassiveMotionFunc(mouseMovimento);    // mouse mexeu
glutTimerFunc(0, timer, 0);               // relógio do jogo

glutMainLoop();                           // entrega o controle ao GLUT (nunca retorna)
```

As quatro funções que formam o jogo

Função	Quando roda	O que faz
`setup()`	uma vez, no início	Liga o teste de profundidade, compila os shaders, configura as luzes, carrega as texturas e monta a cena (velas, portas, colisões).
`timer()`	a cada 16 ms (~60 vezes por segundo)	A **lógica**: calcula o tempo que passou, anima portas, move a câmera, resolve colisões, atualiza a fase do jogo e a mira. No fim pede um redesenho e se reagenda.
`display()`	sempre que um redesenho é pedido	O **desenho**: limpa a tela, posiciona câmera e luzes, desenha todos os objetos, depois o HUD, e troca os buffers.
`keyboard()`	quando uma tecla é apertada	Pausa, interação (E), lamparina (F), reinício (R), wireframe (T)...

A separação importante é **lógica no `timer`, desenho no `display`**. O `timer` muda variáveis (posição da câmera, ângulo da porta); o `display` só lê essas variáveis e desenha.

Double buffer (`GLUT_DOUBLE`): o desenho é feito numa imagem escondida e, quando está pronto, `glutSwapBuffers()` troca a imagem escondida pela visível. Sem isso, o jogador veria a cena sendo desenhada pela metade (a tela piscaria).

2. Transformações e modelagem hierárquica

Ideia central: todo objeto é modelado "pequeno e na origem" e depois **levado ao lugar** com três operações: mover, girar e esticar. Objetos complexos são peças simples, cada uma com suas transformações, **penduradas umas nas outras**.

As três transformações

Função	O que faz	Exemplo no projeto
`glTranslatef(x, y, z)`	Move.	Levar o banco para a sua fileira.
`glRotatef(ang, ex, ey, ez)`	Gira ang graus em volta do eixo (ex, ey, ez).	Abrir a folha da porta em volta do eixo Y.
`glScalef(sx, sy, sz)`	Estica ou encolhe em cada eixo.	Transformar um cubo de 1 m numa tábua.

Essas funções não mexem em nenhum objeto diretamente. Elas alteram a **matriz modelview**, uma "regra de posicionamento" que é aplicada a todos os vértices desenhados dali em diante.

A peça básica: um cubo esticado

ModelosMembroA.h — peca()

```
inline void peca(float x, float y, float z, float largura, float altura, float profundidade) {
    glPushMatrix();                              // guarda a matriz atual
    glTranslated(x, y, z);                       // 2o: leva a peça ao lugar
    glScalef(largura, altura, profundidade);     // 1o: estica o cubo de 1x1x1
    desenharCuboTexturizado(1.0f);
    glPopMatrix();                               // devolve a matriz como estava
}
```

A ordem se lê de baixo para cima: a transformação escrita mais perto do desenho é a primeira a ser aplicada. Aqui o cubo primeiro é esticado e só depois movido. Se fosse o contrário, o `glScalef` esticaria também a distância até a origem e a peça iria parar no lugar errado.

glPushMatrix / glPopMatrix: salvar e restaurar

`glPushMatrix()` tira uma "foto" da matriz atual e guarda numa pilha. `glPopMatrix()` joga fora o que foi feito depois e volta à foto. Isso permite transformar uma peça **sem afetar as próximas**. Tudo que fica entre um push e um pop herda as transformações de fora e acrescenta as suas.

Hierarquia: o banco da igreja

ModelosMembroA.h — desenharBanco() (resumido)

```
glPushMatrix();
glTranslatef(posX, posY, posZ);                 // o banco INTEIRO vai para a sua posição
glRotatef(rotY, 0.0f, 1.0f, 0.0f);

peca(0.0f, 0.45f, 0.0f, C, 0.05f, 0.40f);       // assento
peca(-meio + 0.025f, 0.35f, 0.02f, 0.05f, 0.70f, 0.45f);   // lateral esquerda
...
    glPushMatrix();                             // --- sub-árvore do encosto ---
    glTranslatef(0.0f, 0.475f, 0.18f);          // pivô na dobra
    glRotatef(10.0f, 1.0f, 0.0f, 0.0f);         // inclina 10 graus para trás
    peca(0.0f, 0.275f, 0.0f, C - 0.05f, 0.55f, 0.04f);    // encosto
    peca(0.0f, 0.30f, 0.07f, C - 0.10f, 0.03f, 0.10f);    // porta-livros: HERDA a inclinação
    glPopMatrix();
glPopMatrix();
```

Isso é uma **árvore**: o banco é o pai; assento, laterais e encosto são filhos; o porta-livros é filho do encosto. Como o porta-livros é desenhado dentro do push/pop do encosto, ele se inclina junto, sem nenhuma conta extra. Mover o pai move todos os filhos.

Girar em torno de um ponto: a dobradiça da porta e o pivô do pêndulo

O `glRotatef` sempre gira em torno da **origem atual**. Para girar em torno de uma dobradiça, primeiro se leva a origem até a dobradiça e depois se gira:

ModelosMembroA.h — desenharPortasDuplas() / ModelosMembroC.h — desenharRelogioPendulo()

```
// Folha esquerda da porta: dobradiça em x = -1
glPushMatrix();
glTranslatef(-1.0f, 0.0f, 0.0f);                // origem vai para a dobradiça
glRotatef(angulo, 0.0f, 1.0f, 0.0f);            // gira em volta dela (eixo Y = vertical)
folhaPorta(+1.0f, ...);                         // a folha é modelada a partir da dobradiça
glPopMatrix();

// Pêndulo do relógio: pivô em y = 1.5
glPushMatrix();
glTranslatef(0.0f, 1.5f, 0.21f);                // origem vai para o pivô
glRotatef(anguloPendulo, 0.0f, 0.0f, 1.0f);     // balança em volta do eixo Z
    // haste e disco são desenhados PARA BAIXO do pivô (y negativo) e giram juntos
glPopMatrix();
```

O lustre usa a mesma ideia em escala maior: `glTranslatef` até o ponto do teto, um `glRotatef` de balanço e um `glScalef(escala...)`. Como corrente, aro, braços e velas são desenhados depois, **uma única rotação balança o lustre inteiro**. As 8 velas vêm de um laço que gira 45 graus a cada volta.

3. Câmera e projeção

Ideia central: no OpenGL não existe "câmera" de verdade. Mover a câmera para a direita é o mesmo que mover o mundo inteiro para a esquerda. O `gluLookAt` monta essa transformação; o `gluPerspective` define a "lente".

Yaw e pitch: para onde o jogador olha

A câmera guarda uma posição (`cameraX`, `cameraY`, `cameraZ`) e dois ângulos: **yaw** (girar a cabeça para os lados) e **pitch** (olhar para cima e para baixo). Com seno e cosseno, os dois ângulos viram um vetor de direção de tamanho 1:

CameraMembroA.h — cameraFrente() e cameraAplicar()

```
frenteX = std::cos(y) * std::cos(p);     // y = yaw em radianos, p = pitch em radianos
frenteY = std::sin(p);                   // olhar para cima aumenta o Y
frenteZ = std::sin(y) * std::cos(p);

gluLookAt(cameraX, cameraY, cameraZ,                                    // onde o olho está
          cameraX + frenteX, cameraY + frenteY, cameraZ + frenteZ,      // para onde olha
          0.0f, 1.0f, 0.0f);                                            // qual lado é "para cima"
```

O `gluLookAt` recebe três coisas: a posição do olho, um ponto para onde ele olha (o olho mais a direção) e o vetor "para cima". Com yaw = -90 graus, a direção é (0, 0, -1): o jogador começa olhando para o altar.

Andar com WASD

CameraMembroA.h — atualizarCamera()

```
float frenteX = std::cos(y), frenteZ = std::sin(y);     // direção SEM o pitch: anda no plano do chão
float direitaX = -frenteZ, direitaZ = frenteX;          // perpendicular à frente (90 graus)
float passo = velocidadeCamera * deltaTime;             // 3 m/s x tempo do frame

if (teclas['w']) { cameraX += frenteX * passo;  cameraZ += frenteZ * passo; }
if (teclas['d']) { cameraX += direitaX * passo; cameraZ += direitaZ * passo; }
```

- O movimento ignora o pitch de propósito: olhar para o chão não faz o jogador afundar nele.
- O vetor "direita" é a frente girada 90 graus: trocar (x, z) por (-z, x).
- O vetor `teclas[256]` guarda quais teclas estão apertadas. O teclado só liga e desliga posições; quem move é o `timer`. Por isso dá para andar na diagonal (W + D ao mesmo tempo).

Olhar com o mouse

CameraMembroA.h — cameraMouse()

```
float dx = (float)(x - centroX);          // quanto o mouse saiu do centro da janela
float dy = (float)(centroY - y);
yaw   += dx * sensibilidadeMouse;         // 0.1 grau por pixel
pitch += dy * sensibilidadeMouse;
if (pitch >  89.0f) pitch =  89.0f;       // não deixa "dar cambalhota"
if (pitch < -89.0f) pitch = -89.0f;
glutWarpPointer(centroX, centroY);        // devolve o cursor ao centro
```

A cada movimento, mede-se o quanto o cursor saiu do centro, soma-se aos ângulos e o cursor é devolvido ao centro (`glutWarpPointer`). Assim o mouse nunca bate na borda da tela. O limite de 89 graus existe porque, olhando exatamente para cima, a direção ficaria paralela ao vetor "para cima" e o `gluLookAt` falharia.

Projeção: a lente da câmera

Main.cpp — reshape()

```
glViewport(0, 0, width, height);                // usa a janela inteira
glMatrixMode(GL_PROJECTION);                    // "agora vou configurar a lente"
glLoadIdentity();
gluPerspective(60.0, aspect, 0.1, 100.0);       // abertura, proporção, perto, longe
glMatrixMode(GL_MODELVIEW);                     // volta para "posicionar coisas"
```

Parâmetro	Valor	Significado
campo de visão (fov)	60 graus	Abertura vertical da lente. Maior = "grande angular".
aspect	largura / altura	Proporção da janela. Recalculada ao redimensionar, para a imagem não esticar.
near	0,1 m	Tudo mais perto que isso é cortado.
far	100 m	Tudo mais longe que isso é cortado.

Perspectiva significa que objetos distantes aparecem menores, como no olho humano. A alternativa, a projeção **ortográfica** (`glOrtho`), não tem essa redução e é usada no projeto só para o HUD (seção 8).

As duas matrizes: `GL_PROJECTION` guarda a lente (configurada no `reshape`); `GL_MODELVIEW` guarda a posição da câmera e dos objetos (refeita a cada frame no `display`).

4. Animação com o tempo

Ideia central: animar é mudar um número um pouquinho a cada frame. Para a velocidade ser a mesma em qualquer computador, a mudança é proporcional ao **tempo que passou** (`deltaTime`), não ao número de frames.

Main.cpp — timer()

```
int tempo_atual = glutGet(GLUT_ELAPSED_TIME);                   // milissegundos desde o início
float deltaTime = (tempo_atual - tempo_anterior) / 1000.0f;     // segundos desde o último frame
tempo_anterior = tempo_atual;
```

Com `deltaTime`, "3 metros por segundo" vira `3.0 * deltaTime` metros neste frame. Num computador lento os frames são mais longos e cada passo é maior; no fim, a velocidade é a mesma.

Tipo 1 — ir até um alvo: a porta

LogicMembroC.h — Porta::atualizarAnimacao()

```
float anguloAlvo = aberta ? 90.0f : 0.0f;       // para onde a porta quer ir
float velocidade = 180.0f;                      // graus por segundo

if (anguloAtual < anguloAlvo) {
    anguloAtual += velocidade * deltaTime;      // anda um pouco
    if (anguloAtual > anguloAlvo) anguloAtual = anguloAlvo;     // não passa do alvo
}
```

A tecla E só troca `aberta` entre verdadeiro e falso. Quem move a porta é essa função, um pouco por frame, até chegar a 0 ou 90 graus (meio segundo). No desenho, o ângulo ainda passa por uma suavização (*smoothstep*: `t*t*(3 - 2*t)`), que faz a porta arrancar e parar devagar.

Tipo 2 — vai e volta: pêndulo, lustre e chamas

Main.cpp — display() / ModelosMembroA.h — desenharLustre()

```
float anguloPendulo = std::sin(tempo * 3.0f) * 15.0f;     // oscila entre -15 e +15 graus
float balanco = 3.0f * std::sin(1.2f * tempo);            // lustre: oscila entre -3 e +3 graus
```

O seno sobe e desce entre -1 e 1 para sempre. Multiplicar por fora muda a **amplitude** (o quanto balança); multiplicar o tempo por dentro muda a **velocidade**. Cada lustre recebe o tempo com um atraso (`tempoSeg + i * 0.8f`) para não balançarem todos juntos.

Tipo 3 — disparo único: o golpe da varinha

Ao apagar uma vela, `animVarinha` recebe 0,35 s e o `timer` vai descontando o `deltaTime`. Enquanto for maior que zero, `sin(pi * animVarinha / 0.35)` faz meia onda: sobe de 0 a 1 e volta a 0. Esse valor inclina a ponta da varinha para baixo e de volta.

5. Colisão

Ideia central: cada obstáculo é um **retângulo no chão** (visto de cima) e o jogador é um ponto com uma folga em volta. Antes de aceitar um passo, o jogo pergunta: "nessa nova posição eu estaria dentro de algum retângulo?"

LogicMembroC.h — AABB

```
struct AABB {
    float minX, maxX;
    float minZ, maxZ;

    bool colide(float x, float z, float raioJogador = 0.2f) const {
        return (x + raioJogador > minX && x - raioJogador < maxX &&
                z + raioJogador > minZ && z - raioJogador < maxZ);
    }
};
```

- **AABB** = *Axis-Aligned Bounding Box*, "caixa alinhada aos eixos": um retângulo que não gira, definido só por mínimo e máximo em X e em Z. É a colisão mais simples e mais barata que existe.
- A colisão é **2D** (só X e Z): o jogador não pula nem abaixa, então a altura não importa.
- O `raioJogador` (0,2 m) é a folga: impede que a câmera encoste a ponto de "ver através" da parede.
- A lista de obstáculos é montada à mão em `SistemaColisao::inicializar()`: altar, relógios, blocos de bancos, paredes do confessionário, batente da porta, castiçais do chão. Os limites da igreja são um AABB "ao contrário": o jogador tem de ficar dentro.

Testar um eixo de cada vez: deslizar na parede

LogicMembroC.h — SistemaColisao::processarMovimento()

```
if (!posicaoValida(posNovaX, posAntigaZ, raioJogador)) {   // tenta andar só em X
    posNovaX = posAntigaX;                                 // bateu: cancela só o X
}
if (!posicaoValida(posAntigaX, posNovaZ, raioJogador)) {   // tenta andar só em Z
    posNovaZ = posAntigaZ;                                 // bateu: cancela só o Z
}
```

Se o jogador anda na diagonal contra uma parede, o eixo que bate é cancelado e o outro continua. Resultado: ele **desliza** ao longo da parede em vez de travar. No `timer`, a ordem é: guardar a posição antiga, mover a câmera, e deixar a colisão corrigir a posição nova.

Obstáculos que mudam: as portas

A porta da entrada só bloqueia quando está fechada (ângulo menor que 45 graus). Aberta, o vão fica livre e as duas folhas, agora viradas para dentro, viram dois obstáculos finos (`colisaoPortaEntrada`). As portas do confessionário seguem a mesma regra. Se uma porta fechar com o jogador no vão, ele pode sair: a posição antiga já estava "dentro", então o teste é ignorado.

6. Mira e interação

Ideia central: o jogo descobre o que está no centro da tela comparando duas setas: **para onde o jogador olha** e **do olho até o objeto**. Se apontam quase para o mesmo lado, o objeto está na mira.

LogicMembroC.h — notaMira()

```
float dx = objX - jogX, dy = objY - jogY, dz = objZ - jogZ;      // seta do olho até o objeto
float dist = std::sqrt(dx * dx + dy * dy + dz * dz);
if (dist > alcance || dist < 0.001f) return -1.0f;               // longe demais

float cosAngulo = (dx * fX + dy * fY + dz * fZ) / dist;          // produto escalar
return (cosAngulo >= cosMinimo) ? cosAngulo : -1.0f;             // dentro do "cone" da mira?
```

- O **produto escalar** de dois vetores de tamanho 1 dá o cosseno do ângulo entre eles: 1 = mesma direção, 0 = perpendicular. É a mesma conta N·L da iluminação.
- Exigir cosseno maior que 0,94 equivale a um cone de uns 20 graus em volta do centro da tela.
- A cada frame, `atualizarAlvo()` calcula essa nota para a porta da entrada, as velas acesas e as portas do confessionário, e guarda o objeto de maior nota. A tecla E age nesse objeto; a mira fica amarela e mostra a dica.

7. As fases do jogo

Ideia central: o jogo é uma **máquina de estados**: está sempre em exatamente uma fase, e cada fase tem uma condição para passar à próxima.

GameplayMembroB.h

```
enum class Fase { ApagarVelas, PortaBateu, Esconda, Escondido, FinalRuim, FinalBom };

switch (fase) {
case Fase::ApagarVelas:
    if (velasAcesas(g) == 0) mudar(Fase::PortaBateu);          // apagou a última vela
    break;
case Fase::PortaBateu:
    if (tempoFase >= ESPERA_ESCONDA) mudar(Fase::Esconda);     // passaram 3 segundos
    break;
case Fase::Esconda:
    if (cabine(camX, camZ) != 0) mudar(Fase::Escondido);       // entrou numa cabine
    break;
...
```

Fase	O que o jogador vê	Passa para a próxima quando...
ApagarVelas	"Apague as velas da igreja. Não feche as portas." Lustres acesos.	as 6 velas estão apagadas.
PortaBateu	A porta da entrada fecha e tranca; os lustres apagam. Tela sem texto.	passam 3 segundos.
Esconda	"Esconda-se."	o jogador entra numa cabine do confessionário.
Escondido	Após 3 s o bicho passa. Porta da cabine fechada: vulto lento que entra na cabine do padre. Porta aberta: só uma sombra rápida no chão.	porta fechada: o vulto chega (final ruim). Porta aberta: completam-se 15 s (final bom). Sair da cabine volta para "Esconda".

Fase	O que o jogador vê	Passa para a próxima quando...
FinalRuim / FinalBom	Texto de final e "R para reiniciar".	R recomeça tudo.

A função `mudar(nova)` troca a fase e zera o cronômetro `tempoFase`, que soma o `deltaTime` a cada frame. As outras partes do jogo só **consultam** a fase: o HUD escolhe o texto, as luzes escurecem, a porta tranca, os lustres apagam.

8. HUD: desenhar em 2D por cima da cena

Ideia central: para desenhar texto, mira e menu, o programa **troca a lente** por uma projeção ortográfica em que 1 unidade = 1 pixel, desenha, e depois devolve a lente 3D.

Main.cpp — desenharMira() (o HUD do GameplayMembroB.h segue o mesmo esquema)

```
glMatrixMode(GL_PROJECTION); glPushMatrix(); glLoadIdentity();
glOrtho(0, w, 0, h, -1, 1);                      // tela vira um plano de w x h pixels
glMatrixMode(GL_MODELVIEW); glPushMatrix(); glLoadIdentity();   // sem câmera
glDisable(GL_DEPTH_TEST);                        // o HUD fica sempre por cima

    // ... desenha com glVertex2f(x, y) em pixels ...

glEnable(GL_DEPTH_TEST);
glMatrixMode(GL_PROJECTION); glPopMatrix();      // devolve a perspectiva
glMatrixMode(GL_MODELVIEW);  glPopMatrix();
```

- `glOrtho` é a projeção sem perspectiva. Com (0, w, 0, h), o canto inferior esquerdo é (0, 0) e o superior direito é (w, h).
- O push/pop nas **duas** matrizes garante que a cena 3D do próximo frame não seja afetada.
- Antes do HUD, o `display` chama `glUseProgram(0)`: o shader é desligado, porque texto e mira não devem receber iluminação.
- O texto usa `glRasterPos2f` (posição do primeiro caractere) e `glutBitmapCharacter` (desenha uma letra de uma fonte pronta do GLUT).

9. Profundidade e transparência

Teste de profundidade (z-buffer)

Ideia central: além da cor, cada pixel guarda a **distância** do objeto que foi desenhado ali. Um novo objeto só pinta o pixel se estiver mais perto. Assim a ordem de desenho não importa.

- `glEnable(GL_DEPTH_TEST)` no `setup` liga o teste; `GLUT_DEPTH` no `main` pede o buffer.
- `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)` no início de cada frame limpa a cor e as distâncias.
- Sem o teste, o último objeto desenhado ficaria por cima, mesmo estando atrás de uma parede.

O truque dos objetos na mão

Main.cpp — desenharObjetosNaMao()

```
glLoadIdentity();                   // espaço da câmera: o objeto fica "preso" à tela
glClear(GL_DEPTH_BUFFER_BIT);       // apaga SÓ as distâncias, mantém as cores
// ... desenha a varinha e a lamparina ...
```

A varinha e a lamparina são desenhadas por último, com a matriz identidade (por isso acompanham o olhar). Apagar só o buffer de profundidade faz o OpenGL "esquecer" as distâncias da cena: os objetos da mão nunca são cortados por uma parede próxima, mas as peças deles ainda se escondem corretamente entre si.

Transparência (blending)

GameplayMembroB.h — a sombra do bicho no chão

```
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);   // mistura pela opacidade (alfa)
glDepthMask(GL_FALSE);                               // a sombra não grava profundidade
glBegin(GL_TRIANGLE_FAN);
    glColor4f(0.0f, 0.0f, 0.0f, 0.85f);              // centro: preto, 85% opaco
    glVertex3f(bichoX, 0.02f, bichoZ);
    glColor4f(0.0f, 0.0f, 0.0f, 0.0f);               // borda: totalmente transparente
    ...
```

- O quarto número da cor é o **alfa**: 1 = opaco, 0 = invisível. Com essa `glBlendFunc`, a cor final é **alfa × cor nova + (1 − alfa) × cor que já estava**.
- A sombra é um leque de triângulos com o centro escuro e a borda transparente; a interpolação cria o degradê.
- `glDepthMask(GL_FALSE)` impede que um objeto transparente "tampe" o que for desenhado depois.
- O mesmo blending faz as caixas de texto e o fundo do menu: um retângulo preto com alfa 0,55 a 0,75.

10. Se o professor perguntar: respostas curtas e precisas

Cada resposta tem duas ou três frases e cita a função do código. A ideia é responder direto e, se ele quiser mais, abrir o arquivo indicado.

Estrutura

Pergunta	Como responder
Como o programa funciona?	"Usamos GLUT. No `main` registramos callbacks e entramos no `glutMainLoop`. O `timer` roda a cada 16 ms e cuida da lógica; o `display` só desenha. O `setup` roda uma vez e carrega shaders, texturas e a cena."
Por que double buffer?	"Desenhamos num buffer escondido e trocamos com `glutSwapBuffers` quando o frame está pronto. Isso evita que a tela pisque."

Modelos

Pergunta	Como responder
Como os objetos foram modelados?	"Com primitivas: um cubo texturizado feito à mão com `GL_QUADS`, e cilindros, esferas, cones e toros da GLU/GLUT. Cada objeto é uma composição dessas peças com translação, rotação e escala."
O que é modelagem hierárquica? Onde usaram?	"É organizar o objeto como uma árvore em que cada peça herda as transformações do pai, usando `glPushMatrix`/`glPopMatrix`. No banco, o porta-livros é filho do encosto e se inclina junto. No lustre, uma rotação balança corrente, aro e velas de uma vez."
Como a porta gira na dobradiça?	"Transladamos a origem até a dobradiça e só então rotacionamos em Y. A folha é modelada a partir da dobradiça, então gira em volta dela."
Por que a ordem das transformações importa?	"Porque multiplicação de matrizes não é comutativa. O OpenGL aplica primeiro a transformação escrita mais perto do desenho. Na `peca()`, escalamos o cubo e depois transladamos."
Como representam a malha?	"O cubo usa lista de vértices e lista de faces: 8 posições, e cada face guarda 4 índices. Cada face tem sua normal e seus UVs."

Câmera

Pergunta	Como responder
Como funciona a câmera?	"É em primeira pessoa. Guardamos posição, yaw e pitch. Com seno e cosseno calculamos o vetor de direção e passamos para o `gluLookAt`: olho, olho + direção e o vetor up (0, 1, 0)."
E o mouse?	"Medimos o quanto o cursor saiu do centro da janela, somamos a yaw e pitch e devolvemos o cursor ao centro com `glutWarpPointer`. O pitch é limitado a 89 graus."
Qual projeção usaram?	"Perspectiva, com `gluPerspective`: 60 graus de abertura, near 0,1 e far 100. O aspect é recalculado no `reshape`. Para o HUD trocamos para ortográfica com `glOrtho`."

Texturas

Pergunta	Como responder
Como aplicam as texturas?	"Carregamos a imagem com stb_image e enviamos com `glTexImage2D`. Cada vértice recebe uma coordenada UV com `glTexCoord2f`, e a placa interpola. Usamos `GL_REPEAT`, então UV maior que 1 repete a imagem, como no chão."
E os objetos sem UV?	"As formas do GLUT não geram UV. Para elas ligamos uma textura branca de 1 pixel; como o shader multiplica textura por cor, sobra só a cor do `glColor`."

Iluminação e shaders

Pergunta	Como responder
Qual modelo de iluminação?	"Blinn-Phong com emissão, calculado por pixel no fragment shader (Phong shading). A cor é emissão + ambiente + a soma de difusa e especular de cada luz."
Quais luzes existem?	"Luar direcional, a lamparina na mão, seis velas pontuais nas 8 luzes do OpenGL, e quatro lustres enviados por uniform, porque as 8 vagas estavam ocupadas."
Como a lamparina segue o jogador?	"A posição dela é definida antes do `gluLookAt`, então fica no espaço do olho. As velas são definidas depois e ficam fixas no mundo."
Para que serve o material?	"`materialB(especular, brilho)` define a força e a concentração do reflexo: alto para metal e vidro, baixo para pedra e madeira. A emissão faz chamas brilharem sem luz."

Animação e jogo

Pergunta	Como responder
Como é feita a animação?	"Com `deltaTime`: toda velocidade é multiplicada pelo tempo do frame. A porta anda a 180 graus por segundo até o alvo; pêndulo e lustre usam seno do tempo."
Como funciona a colisão?	"AABB em 2D: cada obstáculo é um retângulo em X e Z, e o jogador tem um raio de 0,2 m. Testamos um eixo de cada vez, então ele desliza na parede. As portas só bloqueiam quando fechadas."
Como sabem para onde o jogador está mirando?	"Produto escalar entre a direção do olhar e a direção até o objeto. Se o cosseno é alto e o objeto está no alcance, ele é o alvo da tecla E."
Como o jogo controla as fases?	"Máquina de estados com um `enum Fase` e um `switch` no `atualizar`. Cada fase tem uma condição de saída: velas apagadas, tempo, entrar na cabine."

HUD, profundidade e transparência

Pergunta	Como responder
Como desenham o texto e a mira?	"Trocamos a projeção para `glOrtho` em pixels, desligamos o shader e o teste de profundidade, desenhamos e restauramos as matrizes com pop."
Como resolvem o que fica na frente?	"Z-buffer: `GL_DEPTH_TEST`. Cada pixel guarda a distância, e só o fragmento mais próximo é mantido."
Por que a lamparina nunca atravessa a parede?	"Antes de desenhar os objetos da mão limpamos só o buffer de profundidade. Eles ficam por cima da cena, mas ainda se ocultam entre si."
Onde usam transparência?	"Na sombra do bicho e nas caixas do HUD, com `GL_BLEND` e `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. A cor é misturada pelo alfa."

Limitações, se ele perguntar o que poderia melhorar

- A colisão é só em 2D e com caixas que não giram; objetos girados (relógios a 45 graus) usam uma caixa aproximada.
- O pêndulo soma um valor fixo por frame (`tempo += 0.016f` no `display`) em vez de usar o `deltaTime`, então a velocidade dele depende da taxa de quadros e ele não para na pausa.
- Não há sombras projetadas: as luzes atravessam os objetos. A "sombra" do bicho é um desenho no chão.
- As texturas não usam mipmaps, o que causa cintilação no chão ao longe.
- Cada lustre é aproximado por uma única luz, não pelas 8 velas.